

Error Handling: `try...catch`

Script me error aaye to wo aam taur par ruk jaati hai ("die"). `try...catch` se hum error pakad kar kuch samajhdaari ka kaam kar sakte hain.

Syntax

```
javascript

try {
  // code chalao
} catch (err) {
  // error aaye to yahan
} finally {
  // hamesha chalta hai
}
```

- Error nahi aaya: `catch` skip.
- Error aaya: `try` wahin ruk jaata hai aur `catch` chalta hai. `err` me error object hota hai.

Sirf runtime errors

- **Syntax galat** ho (jaise `{{{{}}`) to `try...catch` kaam nahi karta (parse-time error).
- **Synchronous** hai: `setTimeout` ke andar ka error bahar wala `try...catch` nahi pakadta, kyunki wo function baad me chalta hai. `try...catch` uske **andar** likhna padega.

Error object

- `name`: error ka naam (jaise `ReferenceError`)
- `message`: description
- `stack`: call stack (debugging ke liye, non-standard par sab jagah)

`catch { ... }` bina `(err)` ke bhi likh sakte ho agar details nahi chahiye.

Real use: `JSON.parse`

Bekaar JSON aaye to `JSON.parse` error deta hai. `try...catch` se user ko sahi message dikha sakte ho, dobara request bhej sakte ho, ya log kar sakte ho.

Apna error: `throw`

JSON sahi ho par `name` field na ho, to bhi hamare liye wo error hai. `throw` se banao:

```
javascript
```

```
if (!user.name) {  
  throw new SyntaxError("Incomplete data: no name");  
}
```

- Kuch bhi throw ho sakta hai, par behtar hai `Error` objects (`Error`, `SyntaxError`, `TypeError`, `ReferenceError`...).
- `new Error(message)` se `name` constructor ka naam hota hai aur `message` argument.

Rethrowing (bahut zaroori)

| `catch` sirf wahi errors sambhale jinhe wo jaanta hai, baaki sab dobara `throw` kar de.

```
javascript  
  
catch (err) {  
  if (err instanceof SyntaxError) {  
    console.log("JSON Error: " + err.message);  
  } else {  
    throw err; // pata nahi, aage bhej do  
  }  
}
```

Warna koi programming galti (jaise variable define nahi) bhi galat message ke saath "JSON Error" ban ke chhup jaayegi.

`finally`

`try` ke baad (ya `catch` ke baad) **har haal me** chalta hai. Cleanup ke liye (loading band karna, connection close karna, time naapna).

```
javascript  
  
try {  
  result = fib(num);  
} catch (err) {  
  result = 0;  
} finally {  
  diff = Date.now() - start; // hamesha  
}
```

- `try` me `return` ho tab bhi `finally` chalta hai (return se just pehle).
- `try...finally` (bina `catch`) bhi valid hai: error bahar nikal jaata hai par cleanup ho jaata hai.

- Variables `try` ke andar `let` se banaoge to sirf wahin dikhenge, isliye bahar declare karo.

Task: `finally` vs seedha code

Function me `return` ya `throw` ho to `try...catch` ke baad likha code chalta hi nahi. `finally` chalta hai. Isliye cleanup ke liye `finally` behtar.

Global catch (environment-specific)

Jo error kisi `try...catch` me nahi aaya use pakadne ke liye:

- Browser: `window.onerror = function(message, url, line, col, error) {...}`
- Node.js: `process.on("uncaughtException")`

Iska kaam script ko recover karna nahi, developers ko error report karna hai (jaise Sentry jaisi services).

Yaad rakho

- `try...catch...finally` runtime errors ke liye.
- `throw` se apne errors banao.
- Unknown error ko rethrow karo.
- Cleanup `finally` me.